

OXYPC INVENTORY LIFECYCLE

PRODUCT

# Personas & Journeys

The 12 roles that handle a device, their journeys through the app, and how a device moves through every stage.

Last updated: 6 Nov 2026 · Source: staging branch @ 55ce58a

App Development Team · Bluemonk

# Personas & Journeys

Last updated: 6 Nov 2026 · Source: staging branch @ 55ce58a

**In this document:** Part A — Personas & Journeys · Part B — Device Lifecycle & Business Process

PART A

# Personas & Journeys

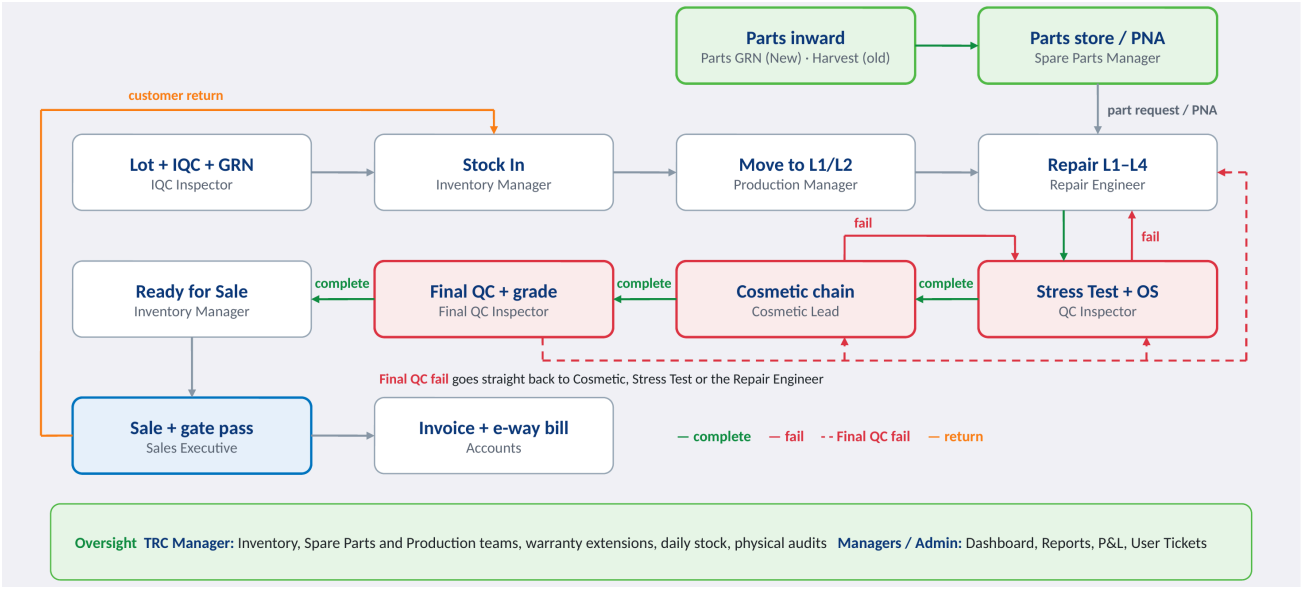
## The 12 personas

Twelve personas cover every role that handles a device, from lot intake to sale and oversight. Each is named by role only. App roles marked `custom role` have no built-in role in the code and are set up through the Permission Matrix (see Access & Security).

| # | Persona | App role | What they do |
|---|---|---|---|
| 1 | **IQC Inspector** | `iqc_inspector` | Brings every purchased lot into the system: creates the lot, inspects each device in IQC, and maps it to a GRN. |
| 2 | **Inventory Manager** | `inventory_manager` | Owns stock outside the repair line: Stock In, floor movement, scrap, Ready for Sale, and holding tags for staff. |
| 3 | **Production Manager** | `custom role` | Runs the repair line from L1/L2 to Final QC: plans lots, moves tags to engineers, reassigns work, and reports team productivity. |
| 4 | **Repair Engineer (L1/L2, L3/L4)** | `l1 / l2 / l3_engineer` | Repairs devices: diagnoses each tag, requests parts, records PNA when a part isn't available, and scraps tags beyond repair. |
| 5 | **QC Inspector (Stress Test)** | `qc_inspector` | Tests every repaired device: runs the stress test, installs the OS, runs system diagnostics, then completes or fails it. |
| 6 | **Cosmetic Lead** | `cosmetic_manager` | Runs the cosmetic chain: Cleaning, Putty, Dry Sanding, Masking, Painting and Water Sanding, then completes or fails each tag. |
| 7 | **Final QC Inspector** | `qc_inspector / custom` | The last check before sale: diagnoses each device by hand and assigns its grade, or fails it back to the stage that needs rework. |
| 8 | **Spare Parts Manager** | `spare_parts_manager` | Runs the parts store: books new parts through Parts GRN and harvested parts from scrap, and fulfils engineers' part requests. |
| 9 | **Sales Executive** | `sales / sales_manager / telecaller` | Sells Ready for Sale stock: blocks tags, submits and records sales, raises delivery challans, and handles returns and rentals. |
| 10 | **Accounts** | `custom role` | Closes the money loop: generates invoices and e-way bills, records payments, and checks the sales list and daily stock. |
| 11 | **Managers / Admin** | `admin / Sub Admin` | Runs the business from the numbers: dashboard, reports and P&L, and resolves the tickets users raise. |
| 12 | **TRC Manager** | `custom role` | Leads the TRC: oversees the Inventory, Spare Parts and Production teams, approves warranty extensions, and owns daily stock and audits. |

## End-to-end device journey

A device passes three quality checks before it can be sold. Stress Test, the Cosmetic chain and Final QC each complete a tag forward or fail it back; Final QC can fail straight back to Cosmetic, Stress Test or the Repair Engineer. Parts come in new through Parts GRN or harvested from old devices. Customer returns come back to Stock In.




## Persona profiles and journeys

One page per persona: profile, goals, frustrations, and the journey through the app, with the module used and the pain point or improvement idea at each step.

# 1. IQC Inspector

**Brings every purchased lot into the system: creates the lot, inspects each device in IQC, and maps it to a GRN.**

1

**Team:** Intake
**App role:** `iqc_inspector`
**Where they work:** Intake bench with a barcode scanner; pallets of 50–200 devices.
**Main modules:** GRN, Lot Overview, IQC Line Items

**GOALS**

- Inspect the whole lot the day it arrives
- Record condition accurately so prices are fair
- No device without a tag

**FRUSTRATIONS**

- Retyping details for identical models
- Bulk-upload errors that are hard to see
- Not knowing which invoice a lot belongs to

## User journey

| Step | What they do | Where in the app | Pain point / idea |
|---|---|---|---|
| **1. Create lot** | Records supplier, invoice and quantity | `Lot Overview` | Invoice details in one place |
| **2. Inspect** | Checks parts and condition of each device | `IQC Line Items` | Repetitive entry for same models |
| **3. Add assets & tags** | Adds assets; assigns Tag Numbers | `IQC Line Items` | Clear bulk-upload error table |
| **4. Map to GRN** | Maps inspected tags to the GRN | `GRN` | One step moves tags to Stock In |
| **5. Hand off** | Tags land in Stock In for Inventory | `Inventory Manager` | Confirm the whole lot moved |

## 2. Inventory Manager

**Owns stock outside the repair line: Stock In, floor movement, scrap, Ready for Sale, and holding tags for staff.**

2

**Team:** Inventory
**App role:** `inventory_manager`
**Where they work:** Desk with the dashboard open; walks the racks when a count looks wrong.
**Main modules:** Stock In, Floor Movement, Ready for Sale, Block Tags, Extended Warranty

**GOALS**

- Stock In moves to production without delay
- Every tag has a known location
- Ready for Sale stock is accurate for sales

**FRUSTRATIONS**

- Tags sitting in Stock In unnoticed
- Floor moves made without a log
- Blocked tags nobody releases

### User journey

| Step | What they do | Where in the app | Pain point / idea |
|---|---|---|---|
| **1. Receive Stock In** | Checks tags arriving from GRN | `Stock In` | Counts match the GRN |
| **2. Move to Production** | Sends tags to the Production Manager | `Stock In → Production` | Bulk move in one step |
| **3. Floor movement** | Moves tags between locations and entities | `Floor Movement` | Every move logged |
| **4. Block for staff** | Holds tags for any staff member's deal | `Block Tags` | Blocks expire or get released |
| **5. Ready for Sale** | Receives graded tags from Final QC | `Ready for Sale` | Grade visible on every tag |
| **6. Scrap & warranty** | Scraps dead tags; extends warranties | `Scrap, Extended Warranty` | Reason recorded for each |

## 3. Production Manager

**Runs the repair line from L1/L2 to Final QC: plans lots, moves tags to engineers, reassigns work, and reports team productivity.**

3

**Team:** Production
**App role:** `custom role`
**Where they work:** On the floor with engineers; checks progress on a laptop through the day.
**Main modules:** Production Manager, Lot Estimation, WorkID Status, Reports

**GOALS**

- Every lot repaired within its budget
- No tag stuck between stages
- Fair, visible productivity per engineer

**FRUSTRATIONS**

- Parts budgets guessed, not estimated
- Learning late that a tag is waiting
- Reassigning work when someone is absent

### User journey

| Step | What they do | Where in the app | Pain point / idea |
|---|---|---|---|
| **1. Estimate lot** | Works out parts needed and repair budget | `Lot Estimation` | Budget set before repair starts |
| **2. Move to L1/L2** | Sends tags from production to engineers | `Production Manager` | Bulk move with assignee |
| **3. Monitor line** | Watches tags from L1/L2 through Final QC | `Production Manager` | One view of every stage |
| **4. Reassign** | Moves work to another engineer | `Production Manager` | Keeps history of who worked |
| **5. Productivity** | Reports output per engineer and team | `WorkID Status, Reports` | Ageing per WorkID, CSV export |

# 4. Repair Engineer (L1/L2, L3/L4)

**Repairs devices: diagnoses each tag, requests parts, records PNA when a part isn't available, and scraps tags beyond repair.**

4

**Team:** Workshop
**App role:** `l1 / l2 / l3_engineer`
**Where they work:** Shared bench PC between repairs, so every extra click costs bench time.
**Main modules:** L1/L2 Repair, L3/L4 Repair, Part Request, PNA

**GOALS**

- Finish more devices per shift
- Get parts without walking to stores
- Credit for every job

**FRUSTRATIONS**

- Part requests with no status
- Waiting on PNA with no update
- Devices failed back with no clear reason

## User journey

| Step | What they do | Where in the app | Pain point / idea |
|---|---|---|---|
| **1. Pick work** | Opens assigned tags; starts a WorkID | `L1/L2 Repair` | Queue in priority order |
| **2. Diagnose & repair** | Finds the fault and fixes it | `L1/L2 Repair` | IQC report and history at hand |
| **3. Part request** | Raises a request for the parts needed | `Part Request` | One row per part, with status |
| **4. PNA** | Submits Part Not Available when stock is out | `PNA Submission` | Alert when the part arrives |
| **5. Escalate** | Sends hard cases up to L3/L4 | `L3/L4 Repair` | Reason travels with the tag |
| **6. Complete or scrap** | Passes to Stress Test, or marks scrap | `L1/L2, L3/L4` | Scrap reason required |

## 5. QC Inspector (Stress Test)

**Tests every repaired device: runs the stress test, installs the OS, runs system diagnostics, then completes or fails it.**

5

**Team:** Quality
**App role:** `qc_inspector`
**Where they work:** Test bench with several devices running tests and OS installs at once.
**Main modules:** Stress Test, OS Installation, System Diagnose

**GOALS**

- No faulty device moves forward
- Clear fail reasons for engineers
- OS installed and checked on every device

**FRUSTRATIONS**

- Repeating the checklist by hand
- Devices returning with the same fault
- Waiting on long tests with no queue view

### User journey

| Step | What they do | Where in the app | Pain point / idea |
|---|---|---|---|
| **1. Receive** | Picks up tags completed in repair | `Stress Test` | Queue with filters like L1/L2 |
| **2. Stress test** | Runs the stress checklist | `Stress Test` | Report saved with the tag |
| **3. OS install** | Installs and activates the OS | `Stress Test` | OS status recorded |
| **4. Diagnose** | Runs system diagnostics | `Stress Test` | Results attached to the tag |
| **5. Complete or fail** | Moves forward to Cosmetic, or fails back | `Stress Test` | Fail reasons go with the tag |

# 6. Cosmetic Lead

**Runs the cosmetic chain: Cleaning, Putty, Dry Sanding, Masking, Painting and Water Sanding, then completes or fails each tag.**

6

**Team:** Cosmetic
**App role:** `cosmetic_manager`
**Where they work:** Leads a crew of workers moving tags between stations all day.
**Main modules:** Cosmetic Received, Cosmetic Chain, Devices Passed

**GOALS**

- Batches move between stations quickly
- Fair credit for each worker
- Finish quality that passes Final QC

**FRUSTRATIONS**

- Moving tags one by one
- Credit to whoever clicked, not who worked
- Failed tags with no reason

## User journey

| Step | What they do | Where in the app | Pain point / idea |
|---|---|---|---|
| **1. Cleaning** | Cleans the body and screen | `Cosmetic Chain` | Batch assign to one worker |
| **2. Putty** | Fills dents and scratches | `Cosmetic Chain` | Mover credited automatically |
| **3. Dry sanding** | Smooths the putty | `Cosmetic Chain` | Selections kept while searching |
| **4. Masking** | Masks ports, logos and screens | `Cosmetic Chain` | Tag Number shown first |
| **5. Painting** | Paints the body | `Cosmetic Chain` | Station time visible |
| **6. Water sanding** | Final finish; complete or fail | `Cosmetic Chain` | Complete → Final QC, or fail back |

## 7. Final QC Inspector

**The last check before sale: diagnoses each device by hand and assigns its grade, or fails it back to the stage that needs rework.**

7

**Team:** Final QC
**App role:** `qc_inspector / custom`
**Where they work:** Final QC desk, checking each device in person before it reaches Ready for Sale.
**Main modules:** Final QC, Grading, Search Tags

**GOALS**

- Only sale-ready devices pass
- Grades that match the device
- Failed tags reach the right stage

**FRUSTRATIONS**

- Grade rules that differ between people
- Missing part and repair history
- Deciding which stage a fail goes back to

### User journey

| Step | What they do | Where in the app | Pain point / idea |
|---|---|---|---|
| **1. Receive** | Picks up tags completed in Cosmetic | `Final QC` | Full history on one screen |
| **2. Manual diagnose** | Checks the system by hand | `Final QC` | Checklist by model |
| **3. Assign grade** | Grades the device | `Final QC` | Grade rules shown beside the field |
| **4. Fail back** | Sends to L1/L2, Stress or Cosmetic | `Final QC` | Pick the stage and the reason |
| **5. Complete** | Moves to Ready for Sale (Inventory) | `Final QC → Ready for Sale` | Grade travels with the tag |

# 8. Spare Parts Manager

**Runs the parts store: books new parts through Parts GRN and harvested parts from scrap, and fulfils engineers' part requests.**

8

**Team:** Stores
**App role:** `spare_parts_manager`
**Where they work:** Between the store counter and the racks; engineers come here when a repair stalls.
**Main modules:** Parts Manager, Parts GRN, Part Requests

**GOALS**

- Fill every request the same day
- Use harvested parts before buying new
- Exact stock per part and shelf

**FRUSTRATIONS**

- Requests bundling several parts in one row
- Harvested parts not booked
- PNA requests with no follow-up

## User journey

| Step | What they do | Where in the app | Pain point / idea |
|---|---|---|---|
| **1. Parts GRN** | Books As New parts from vendors | `Parts GRN` | Qty and purchase price captured once (price shown to management only) |
| **2. Harvest inward** | Books As Harvest parts from scrapped tags | `Parts Manager` | Harvest linked to its source tag |
| **3. Fulfil requests** | Works through engineers' requests | `Parts Manager` | One row per part |
| **4. Issue** | Issues parts; consumption recorded | `Parts Consumption` | Date filter covers consumption |
| **5. Close PNA** | Fills PNA requests when parts arrive | `Parts Manager` | Alert to the engineer |

# 9. Sales Executive

**Sells Ready for Sale stock: blocks tags, submits and records sales, raises delivery challans, and handles returns and rentals.**

9

**Team:** Sales
**App role:** `sales / sales_manager / telecaller`
**Where they work:** Calls with dealers and customers, working from the Ready for Sale list.
**Main modules:** Ready for Sale, Block Tags, Sales, Gate Pass, Returns, Rentals

**GOALS**

- Answer 'do you have this model?' in seconds
- Never sell a held tag
- Returns resolved the right way, fast

**FRUSTRATIONS**

- Stock that looks free but is blocked
- Retyping sale details for the challan
- Choosing the right return type

## User journey

| Step | What they do | Where in the app | Pain point / idea |
|---|---|---|---|
| **1. Check stock** | Filters Ready for Sale by model and grade | `Ready for Sale` | Fast filters and exports |
| **2. Block** | Holds tags for a deal | `Block Tags` | Blocked tags clearly marked |
| **3. Sell** | Submits the sale; marks items sold | `Sales, Sales Report` | The only way into Sold |
| **4. Dispatch** | Raises the delivery challan / gate pass | `Gate Pass` | Challan from the sale record |
| **5. Returns** | Return, paid repair, replacement, or credit note | `Returns, Credit Note` | Type decides where the tag goes |
| **6. Rentals** | Rents out devices and tracks them | `Rentals` | Due dates always visible |

# 10. Accounts

**Closes the money loop: generates invoices and e-way bills, records payments, and checks the sales list and daily stock.**

10

**Team:** Finance
**App role:** `custom role`
**Where they work:** Accounts office, working from sales, bank statements and the app.
**Main modules:** Invoices, E-way Bill, Payments, Sales List, Daily Stock

**GOALS**

- Invoice and e-way bill out with every dispatch
- Every sale matched to a payment
- Daily stock that matches the books

**FRUSTRATIONS**

- Retyping sale details into invoices
- E-way bills generated outside the app
- Payments with no sale reference

## User journey

| Step | What they do | Where in the app | Pain point / idea |
|---|---|---|---|
| **1. Invoice** | Generates the invoice for each sale | `Invoices` | Built from the sale record |
| **2. E-way bill** | Generates the e-way bill for dispatch | `E-way Bill Generator` | Ready before the vehicle leaves |
| **3. Payments** | Records customer and supplier payments | `Payments` | Outstanding per customer |
| **4. Sales list** | Checks sales against invoices and payments | `Sales List` | Paid / due on every sale |
| **5. Daily stock** | Reviews opening and closing stock | `Daily Stock` | Matches the books |

# 11. Managers / Admin

**Runs the business from the numbers: dashboard, reports and P&L, and resolves the tickets users raise.**

11

**Team:** Management
**App role:** `admin / Sub Admin`
**Where they work:** Reads dashboards and reports daily; reviews P&L weekly.
**Main modules:** Dashboard, Reports, P&L, User Tickets

**GOALS**

- Stock, cash and profit by lot at any time
- Slow stock and margin leaks spotted early
- User problems resolved quickly

**FRUSTRATIONS**

- Dashboard numbers that disagree with pages
- Reports exported and reworked in Excel
- Tickets with no owner

## User journey

| Step | What they do | Where in the app | Pain point / idea |
|---|---|---|---|
| **1. Dashboard** | Stock by stage, money and weekly trends | `Overall Dashboard` | Tiles match their pages |
| **2. Reports** | Sales, stock and team reports | `Reports` | Excel export for meetings |
| **3. P&L** | Profit by financial year and by lot | `Business P&L, Lot P&L` | Lot margin at a glance |
| **4. User tickets** | Reviews and resolves raised tickets | `Ticket Queue` | Owner and status on each |

# 12. TRC Manager

**Leads the TRC: oversees the Inventory, Spare Parts and Production teams, approves warranty extensions, and owns daily stock and audits.**

12

**Team:** TRC
**App role:** `custom role`
**Where they work:** Moves between teams through the day; reviews numbers each morning and evening.
**Main modules:** Inventory, Spare Parts, Production, Extended Warranty, Daily Stock, Physical Audits

**GOALS**

- Teams hand off without delays
- Daily stock and audits always clean
- Warranty extensions handled fairly

**FRUSTRATIONS**

- Problems surfacing only at audit
- Three teams' status in three places
- Warranty requests with no history

## User journey

| Step | What they do | Where in the app | Pain point / idea |
|---|---|---|---|
| **1. Daily stock** | Reviews opening and closing stock by stage | `Daily Stock` | Exceptions flagged first |
| **2. Inventory team** | Checks Stock In, floor moves, Ready for Sale | `Inventory Manager` | Ageing per stage |
| **3. Spare Parts team** | Checks requests, PNA and parts stock | `Parts Manager` | Open PNA count |
| **4. Production team** | Checks the repair line and productivity | `Production Manager` | Bottleneck stage highlighted |
| **5. Warranty** | Approves warranty extensions | `Extended Warranty` | Device history on the request |
| **6. Audits** | Runs physical and zone audits | `Physical Audits` | Gaps listed by location |

PART B

# Device Lifecycle & Business Process

## 1. Purpose

This document describes how one device (one **Tag Number**, stored as `devices.barcode`) moves through the OxyPC Inventory Lifecycle app. It is written from the code, not from the business view. Where the two differ, the gap is listed in section 9.

Use it to:

- see every stage value the app stores in `devices.current_stage`;
- find which screen and endpoint moves a tag from one stage to another;
- check the guards (what must be true) and side effects (WorkIDs, part requests, audit rows) of each move;
- understand the smaller state machines that run beside the main one (repair status, cosmetic chain, Final QC holds, part requests, PNA, returns, credit notes, block tags).

Related: Business Rules & SOPs · Role & Permission Matrix.

### 1.1 How stage moves are recorded

| Record | What it holds | Where |
|---|---|---|
| `devices.current_stage` | The one current stage (enum `DeviceStage`) | `models/device.py::Device` |
| `stage_movements` | One row per move: from, to, who, when, note. `exited_at` is stamped on the previous open row when the tag leaves a stage | `models/device.py::StageMovement` |
| `work_orders` | One row per assignment to a person (the **WorkID**). Status `pending` → `in_progress` → `completed` | `models/work_order.py::WorkOrder` |
| `audit_logs` | Business audit rows written by `services/audit_engine.py::audit` (not every move writes one) | `services/audit_engine.py` |
| `allowed_transitions` | Admin-editable table of legal moves. Only some endpoints check it (see 1.2) | `models/stage_control.py`, `routers/stage_control.py` |

**WorkID formats** (from `routers/transfers.py::_gen_work_id` and `routers/repair.py::_gen_prefixed_work_id`):

| WorkID | Meaning | `work_orders.stage` code |
|---|---|---|
| 12-digit number | Normal assignment (pick, bucket assign, cosmetic move, Final QC decision, Stress result) | `l1`, `qc`, `strs`, `recv`, `clean`, `putty`, `drysd`, `mask`, `paint`, `water`, `comp`, `fqc`, `asgn`, or first 5 letters of the stage |
| `L3L4-0000001` | L3/L4 repair request | `l3` |
| `STRS-0000001` | Hand-off from L1/L2 Mark Complete to Stress Test | `qc` |
| `L1L2-…` | Legacy L1/L2 WorkIDs (still closed by `_close_l1l2_work_orders`) | `l1` / `l2` |

### 1.2 The Control Engine

`services/control_engine.py` has four checks:

| Function | Rule | Used by |
|---|---|---|
| `assert_device_in_stage` | Tag must be in the expected stage, else HTTP 409 "STAGE CONFLICT" | `routers/repair.py::start_repair`, `complete_repair`; `routers/qc.py::qc_submit` |
| `validate_transition` | Move must exist in `allowed_transitions`, else HTTP 403. Admin bypasses | `routers/repair.py::start_repair`, `complete_repair`, `move_device`; `routers/qc.py::qc_submit`; `routers/grn.py::grn_bulk_move_to_trc`; `routers/iqc.py::iqc_bulk_apply_grade_type`; `routers/bulk_upload.py::approve_duplicate_move`; `routers/api_v1/devices.py::move_device_stage` |
| `validate_sale_allowed` | Refuses a sale only when `devices.is_blocked` is true. Since 19 Oct 2026 a sale is allowed **from any stage** | `routers/sales.py::create_sale` |
| `validate_repair_level` | L2 needs an L1 RepairJob; L3 needs an L2 RepairJob | `routers/repair.py::start_repair` |

Most dedicated workflow screens (L1/L2 Mark Complete, Stress Test, Cosmetic, Final QC, buckets, returns) **do not** call `validate_transition`. Their own code decides the next stage.

## 2. Stage reference

Stage values come from `models/device.py::DeviceStage`. Display names come from `STAGE_LABELS`. Badge colours come from `STAGE_COLORS` (Bootstrap colour names; `teal` and `purple` are custom).

Owner role = the business persona that works the stage. The code role in brackets is the built-in `UserRole` that the code actually checks.

| # | Code value | Display name | Colour | Owner role (code role) | Entry action(s) | Exit action(s) |
|---|---|---|---|---|---|---|
| 1 | `grn` | GRN Receipt | warning | IQC Inspector (any signed-in user) | Register tag on lot: `routers/stock.py::register_device_barcode`, `register_bulk_csv` | Advance GRN to IQC → `iqc` |
| 2 | `iqc` | IQC | secondary | IQC Inspector (`iqc_inspector`) | IQC form without GRN no. (`routers/iqc.py::iqc_create`); OxyQC agent (`routers/iqc_api.py::submit_iqc`); GRN Records "Sending for Selling"; return approved; External Tag return creates new tag | Map GRN / Validate GRN → `stock_in`; Customise move |
| 3 | `stock_in` | Stock In | info | Inventory Manager (`inventory_manager`) | GRN mapped/validated; IQC form with GRN no.; bucket create; Back to Inventory from repair; return submitted; manual moves | Move to Production → `trc_production`; Move to L1/L2 → `l1`; Customise / manual moves |

| # | Code value | Display name | Colour | Owner role (code role) | Entry action(s) | Exit action(s) |
|---|---|---|---|---|---|---|
| 4 | `trc_production` | TRC Production | info | Production Manager ( `inventory_manager` , or custom role `production_manager` ) | Move to TRC / Move bucket to Production; Final QC Fail (Hardware, no engineer found); Change Engineer | Assign / Bulk Assign / Change Engineer → `l1` , `qc_check` , `cosmetic_received` |
| 5 | `l1` | L1/L2 Repair | warning | Repair Engineer L1/L2 ( `l1_engineer` , `l2_engineer` ) | Move to L1/L2; bucket / device assign; Stress fail; Cosmetic fail; Final QC fail (Hardware); L3/L4 complete or scrap decision | Request to L3/L4 → `l3` ; Mark Complete → `qc_check` ; Back to Inventory → `stock_in` or `scrap_for_sale` ; Scrap → `scrapped` |
| 6 | `l2` | L2 Repair | warning | Legacy — no page | None today (retired; kept for old rows) | Legacy `complete_repair` moves |
| 7 | `l3` | L3/L4 Repair | danger | Repair Engineer L3/L4 ( `l3_engineer` ) | Request to L3/L4 from L1/L2; Change Engineer | L3/L4 Complete or Scrap → back to `l1` |
| 8 | `qc_check` | Stress Test | primary | QC Inspector ( `qc_inspector` ) | L1/L2 Mark Complete; bucket assign "Stress Test"; Final QC fail (Software); Change Engineer | Complete → `cosmetic_received` ; Fail → `l1` |
| 9 | `cosmetic_received` | Cosmetic Received | teal | Cosmetic Lead ( `cosmetic_manager` ) | Stress Complete; bucket assign "Cosmetic Repair"; Final QC fail (Cosmetic); Send to Cosmetic | Move → `cleaning` ; Skip → `final_qc` ; Fail → `l1` |
| 10 | `cleaning` | Cleaning | teal | Cosmetic Lead / stage user | Move from Cosmetic Received; bulk Assign | Move → `putty` ; Skip → `final_qc` ; Fail → `l1` |
| 11 | `putty` | Putty | teal | Cosmetic Lead / stage user | Move from Cleaning | Move → `dry_sanding` ; Skip → `final_qc` |
| 12 | `dry_sanding` | Dry Sanding | teal | Cosmetic Lead / stage user | Move from Putty | Move → `masking` ; Skip → `final_qc` |
| 13 | `masking` | Masking | teal | Cosmetic Lead / stage user | Move from Dry Sanding | Move → `painting` ; Skip → `final_qc` |
| 14 | `painting` | Painting | teal | Cosmetic Lead / stage user | Move from Masking | Move → `water_sanding` ; Skip → `final_qc` |
| 15 | `water_sanding` | Water Sanding | teal | Cosmetic Lead / stage user | Move from Painting | Move → `cosmetic_completed` ; Skip → `final_qc` ; Fail → `l1` |
| 16 | `cosmetic_completed` | Cosmetic Completed | teal | Cosmetic Lead | Move from Water Sanding | Move → `final_qc` ; Fail → `l1` |

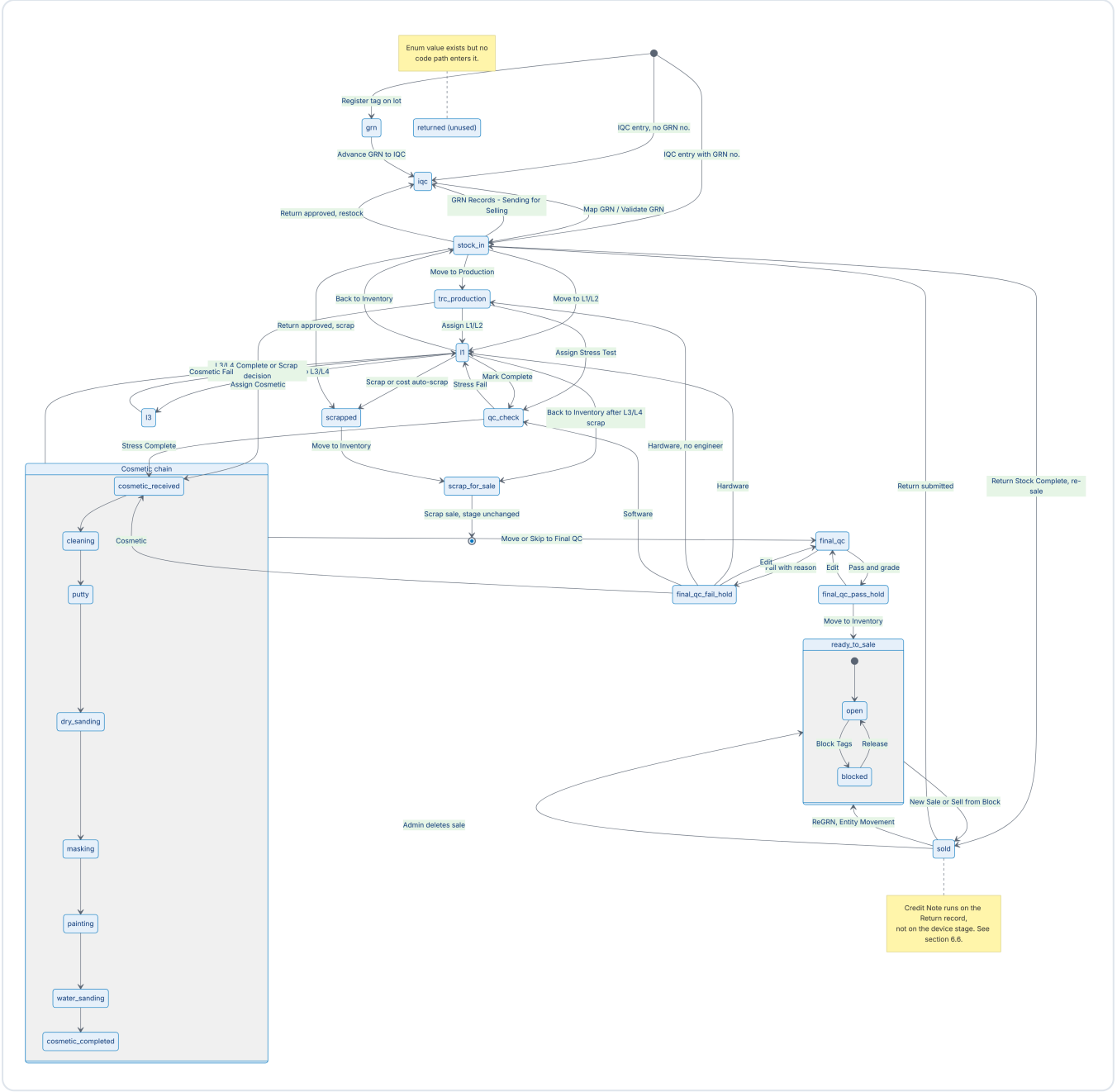
| # | Code value | Display name | Colour | Owner role (code role) | Entry action(s) | Exit action(s) |
|---|---|---|---|---|---|---|
| 17 | `final_qc` | Final QC | purple | Final QC Inspector (any role with `cosmetic_finalqc` edit permission) | From cosmetic chain; Edit (revert) from a hold; Change Engineer | Pass → `final_qc_pass_hold` ; Fail → `final_qc_fail_hold` |
| 18 | `final_qc_pass_hold` | Final QC — Pass Hold | success | Final QC Inspector → Inventory Manager | Final QC Pass | Move to Inventory → `ready_to_sale` ; Edit → `final_qc` |
| 19 | `final_qc_fail_hold` | Final QC — Fail Hold | warning | Final QC Inspector | Final QC Fail | Move / Bulk Move → `l1` , `trc_production` , `qc_check` or `cosmetic_received` ; Edit → `final_qc` |
| 20 | `ready_to_sale` | Ready to Sale | success | Inventory Manager / Sales Executive | Move to Inventory from Pass Hold; ReGRN of a sold tag; Trade Partner test devices; Customise | Sale → `sold` |
| 21 | `sold` | Sold | dark | Sales Executive ( `sales` , `sales_manager` , `telecaller` ) | New Sale / New Tag Sale / Sell from Block; Return Replace Now; Return Stock Complete | Return submitted → `stock_in` ; ReGRN → `ready_to_sale` ; Sale deleted → previous stage |
| 22 | `returned` | Returned | warning | — | **No code path sets this stage today** (see Q2) | Allowed transitions list `iqc` , `scrapped` , `stock_in` |
| 23 | `scrapped` | Scrapped | danger | Inventory Manager / Production Manager | Repair "Scrap" or cost auto-scrap ( `services/cost_engine.py::auto_scrap_device` ); return approved with action Scrap | "Move to Inventory" → `scrap_for_sale` |
| 24 | `scrap_for_sale` | Scrap for Sale | dark | Inventory Manager (Tags Scrapped page) | Back to Inventory after L3/L4 scrap; Scrap Products "Move to Inventory"; Customise | None in code (scrap sale does not change stage, see Q6) |

Notes:

- `DROPDOWN_STAGES` (all stages except `l2` and `sold` ) feeds every generic "move to any stage" dropdown ( `models/device.py` ).
- `COSMETIC_STAGES` groups stages 9–16 for dashboard counts.
- `stage_sort_expr` sorts stages in the enum order above.
- `is_blocked` , `return_status` , `tag_return_status` , `final_qc_status` , `l1l2_status` and `l34_status` are **flags beside the stage**, not stages.

## 3. Lifecycle diagram

The diagram shows the main paths. Generic admin/Customise moves (any stage to any stage) are left out to keep it readable; see transition T40–T43.




# 4. Transitions

"Role" shows the code guard first, then the business persona. "Any user" means the endpoint only checks that someone is signed in ( `get_current_user` ).

## 4.1 Intake

| # | From | To | Trigger / screen | Role | Guards | Side effects |
|---|---|---|---|---|---|---|
| T01 | (new) | `grn` | Lot → Register Device / Bulk CSV (`routers/stock.py::register_device_barcode`, `register_bulk_csv`) | Any user · IQC Inspector | Lot exists | Device row created |
| T02 | `grn` | `iqc` | Lot → Advance GRN to IQC (`routers/stock.py::advance_grn_to_iqc`) | Any user · IQC Inspector | Only tags at `grn` in that lot | **No StageMovement, no audit** (Q12) |
| T03 | (new) | `iqc` or `stock_in` | IQC form (`routers/iqc.py::iqc_create`) | `admin`, `inventory_manager`, `iqc_inspector`, `sales_manager` · IQC Inspector | Barcode must be new; lot must resolve (blank lot → "Unassigned" lot via `utils/lot_helpers.get_or_create_unassigned_lot`). GRN no. given → `stock_in`, else `iqc` | IQCInspection row; StageMovement "IQC Entry" (+ second row to Stock In if GRN given); `device_price` from lot line item or lot average; audit `DEVICE_IQC_REGISTERED` |
| T04 | (new) | `iqc` | OxyQC agent submit (`routers/iqc_api.py::submit_iqc`), API (`routers/api_v1/iqc.py::register_device`) | API / agent | — | Device + inspection rows |
| T05 | `iqc` | `stock_in` | GRN Post-IQC → Map this GRN (`routers/grn.py::grn_map`) | `admin`, `inventory_manager` · Inventory Manager (business: IQC Inspector) | Only tags at `iqc` move; others just get the GRN no. | `grn_number` set; blank entity → "OxyPC Computers"; StageMovement; default location; GRN Stocked memory; audit `GRN_MAPPED` |
| T06 | `iqc` | `stock_in` | GRN Import → Validate (`routers/grn.py::grn_validate`) | `admin`, `inventory_manager` | GRN linked to a lot; tags of that lot at `iqc` | GRN marked validated; auto-map as T05. No audit row |
| T07 | any (allowed) | `iqc` / `stock_in` | GRN Records → Pending for TRC bulk move (`routers/grn.py::grn_bulk_move_to_trc`) | `admin`, `inventory_manager` | `validate_transition` (admin bypass); failed tags skipped | "Selling" → `iqc` and GRN cleared; "Sold to OxyPC" → `stock_in`; audit `GRN_BULK_MOVE_TO_TRC` |

## 4.2 Stock and production

| # | From | To | Trigger / screen | Role | Guards | Side effects |
|---|---|---|---|---|---|---|
| T08 | any | `stock_in` | Create bucket (`routers/buckets.py::create_bucket`) | `admin`, `inventory_manager` · Inventory Manager | Active, not trashed. **No stage guard** | Bucket created or reused by name; location set. **No StageMovement** (Q12) |
| T09 | `stock_in` | `trc_production` | Move to TRC (`routers/stock.py::move_to_trc`) | `admin`, `inventory_manager`, `qc_inspector`, `sales_manager` · Inventory Manager | Tag must be at `stock_in` | StageMovement |
| T10 | `stock_in` (or any, for customer-return buckets) | `trc_production` | Bucket → Move to Production / Assign to Production (`routers/buckets.py::move_bucket_to_trc`, `assign_bucket_to_production` → `_move_bucket_devices_to_trc`) | `admin`, `inventory_manager` | Only `stock_in` tags, unless `bucket.is_customer_return` | Bucket status `trc_pending`, `assigned_to_production`; StageMovement per tag |
| T11 | `stock_in` | `l1` | Move to L1/L2, single or bulk (`routers/stock.py::move_to_l1l2`, `bulk_move_to_l1l2` → `_move_device_to_l1l2`) | Stock roles · Inventory Manager | Tag must be at `stock_in` | Stale open `l1` WorkOrders closed; repair status reset to New (`reset_repair_cycle_on_entry`); lands **unassigned** (Pick This) |
| T12 | any | `l1` / `qc_check` / `cosmetic_received` | Production Manager → Assign bucket / Assign device / Bulk Assign (`routers/buckets.py::assign_bucket`, `assign_device`, `bulk_assign_devices_l1l2` → `_apply_department_move`) | `admin`, `inventory_manager` (+ custom `production_manager`) · Production Manager | Department must map in `DEPT_TO_STAGE` ("L1/L2 Repair", "Stress Test", "Cosmetic Repair") | StockTransfer (`department_assignment`); open WorkOrders closed (`services/work_order_close.py::close_open_work_orders`); new WorkID or reuse of open unassigned STRS- (`reuse_open_stress_work_order`); notification; status reset on L1/L2 entry |
| T13 | any | `l1` | Bucket → Assign to Engineer (`routers/buckets.py::assign_bucket_to_engineer`) | `admin`, `inventory_manager` | — | StageMovement; status reset; bucket `validated` |

| # | From | To | Trigger / screen | Role | Guards | Side effects |
|---|---|---|---|---|---|---|
| T14 | any active | `l1`, `l3`, `qc_check`, `cosmetic_received`, `final_qc`, `stock_in`, `trc_production` | Change Engineer (`routers/stock.py::change_engineer`) | Stock roles · Production Manager | Target in `CHANGE_ENGINEER_STAGES`; user active. **No source-stage guard** | Close open WorkOrders, new WorkID (or STRS- reuse); notification; audit `change_engineer` |
| T15 | any | `stock_in` | Move to Stock (`routers/stock.py::move_to_stock`) | Stock roles | **No stage guard** | StageMovement; flag cleared by `_clear_flag_on_stock_in` |

## 4.3 Repair (L1/L2 and L3/L4)

| # | From | To | Trigger / screen | Role | Guards | Side effects |
|---|---|---|---|---|---|---|
| T16 | `l1` | `l1` | Pick This (`routers/repair.py::l1_pick`) | Permission `repair_l1` add · Repair Engineer | Tag at `l1`; picking own tag is a no-op | All open WorkOrders closed; new 12-digit WorkID; same-stage StageMovement; previous engineer notified; audit `L1L2_PICKED` |
| T17 | `l1` | `l1` | Start Repair (`routers/repair.py::l1l2_start`) | Any user | — | `l1l2_status` = "Repair Started"; RepairJob opened; audit |
| T18 | `l1` | `l3` | Request to L3/L4 (`routers/repair.py::request_l3l4`) | Any user · Repair Engineer L1/L2 | L3/L4 engineer must exist | `l1l2_status` = "Requested to L3/L4"; optional `repair_notes`; new `L3L4-` WorkID (L1/L2 WorkID left open); notification; audit `REQUEST_L3L4` |
| T19 | `l3` | `l3` | L3/L4 Pick / Start (`routers/repair.py::l3l4_pick`, `l3l4_start`) | `repair_l3` enable (pick) / any user (start) | Tag at `l3` (pick) | Pick: close open WorkIDs, new `L3L4-` WorkID. Start: `l34_status` = "Repair Started" |
| T20 | `l3` | `l1` | L3/L4 Mark Complete (`routers/repair.py::l3l4_complete`) | Any user · Repair Engineer L3/L4 | — | `L3L4-` WorkOrders completed; `l34_status` = "Completed"; `l1l2_status` = "Returned from L3/L4"; activity + notes; audit |
| T21 | `l3` | `l1` | L3/L4 Scrap (`routers/repair.py::l3l4_scrap`) | Any user · Repair Engineer L3/L4 | Type must be "Normal Scrap" or "Replacement Scrap" | `l34_status` = scrap type; L1/L2 page then shows "Back to Inventory" |

| # | From | To | Trigger / screen | Role | Guards | Side effects |
|---|---|---|---|---|---|---|
| T22 | `l1` (any) | `scrap_for_sale` | Back to Inventory after scrap decision ( `routers/repair.py::back_to_production` ) | Any user · Repair Engineer | **No stage or status guard** | `grade` = scrap; RepairJobs closed; audit `BACK_TO_INVENTORY` . Tag shows on Tags Scrapped page |
| T23 | `l1` (any) | `qc_check` | L1/L2 Mark Complete ( `routers/repair.py::l1l2_complete_to_stress` ) | Any user · Repair Engineer | **Blocked if any active PNA part**; no stage guard | Open part requests: `handed_over` → `received` (cost counted), `requested` → `cancelled` ; L1/L2 WorkOrders and RepairJobs closed; `l1l2_status` = "Completed"; new `STRS-` WorkID (engineer optional); audits `PART_AUTO_VERIFIED_ON_COMPLETE` , `PART_AUTO_CANCELLED_ON_COMPLETE` , `L1L2_COMPLETE_TO_STRESS` |
| T24 | `l1` | `l1` / `qc_check` / `stock_in` / `scrapped` | Legacy side panel "Complete Job" ( `routers/repair.py::complete_repair` ) | Permission `repair_<stage>` edit | `assert_device_in_stage` ; `validate_transition` on some branches | Final status decides: Scrap or cost auto-scrap → `scrapped` ; Escalate → stays `l1` ; PNA → no move; Back to Inventory → `stock_in` ; Completed/OK → `qc_check` (no STRS- WorkID). Writes RepairAttempt, parts consumption, `REPAIR_COMPLETE` audit |
| T25 | `stock_in` / `l1` | `l1` | Legacy Start Repair ( `routers/repair.py::start_repair` ) | Permission `repair_l1` add | Stage check, repair-level check, transition check | RepairJob opened |

## 4.4 Stress Test, cosmetic and Final QC

| # | From | To | Trigger / screen | Role | Guards | Side effects |
|---|---|---|---|---|---|---|
| T26 | `qc_check` (any) | `l1` | Stress Test → Fail ( `routers/stress_api.py::stress_fail_assign` ) | Any user · QC Inspector | Engineer must have role `l1_engineer` or `l2_engineer` . **No stage guard** | `stress_notes` = failed checklist items + note; status reset; checklist result + PDF saved; completed `strs` WorkID for the tester; new `l1` WorkID for engineer; notification; audit `STRESS_FAIL_ASSIGNED` |
| T27 | `qc_check` (any) | `cosmetic_received` | Stress Test → Complete ( `routers/stress_api.py::stress_complete_to_paint` ) | Any user · QC Inspector | Optional engineer must be `qc_inspector` , `inventory_manager` , `sales_manager` or `cosmetic_manager` | `stress_notes` cleared; checklist + PDF; completed `strs` WorkID; new `recv` WorkID (may be unassigned); audit `STRESS_COMPLETE_TO_PAINT` |

| # | From | To | Trigger / screen | Role | Guards | Side effects |
|---|---|---|---|---|---|---|
| T28 | `qc_check` | `cosmetic_received` or `l1` / `l2` / `l3` | Legacy QC form ( `routers/qc.py::qc_submit` ) | `qc_check` add permission | `assert_device_in_stage` ; `validate_transition` | QCCheck row with scores; pass sets grade; 3rd fail notes "recommend scrap" |
| T29 | any | `cosmetic_received` | Send to Cosmetic ( `routers/cosmetic.py::send_to_cosmetic` ) | Cosmetic roles + `cosmetic_received` add | **No stage guard** | StageMovement only |
| T30 | cosmetic stage *n* | stage *n*+1 | Cosmetic page → Move ( `routers/cosmetic.py::advance_stage` ) | Cosmetic roles + per-stage edit permission ( `PERM_MODULE_BY_STAGE` ) · Cosmetic Lead | Stage must be in `NEXT_COSMETIC` . Assignee required unless landing on `final_qc` or `cosmetic_completed` ; Flow Data can auto-pick ( `_resolve_flow_next_user` ) | Origin stage's pending WorkOrder completed; new WorkID for assignee; notification. **No audit row** |
| T31 | any cosmetic stage | `final_qc` | Skip cosmetic ( `advance_stage` with `target=final_qc` ) | As T30 | Tag in `COSMETIC_PIPELINE` | `bucket_id` cleared on arrival at Final QC |
| T32 | `cosmetic_received` | `cleaning` | Bulk Assign ( `routers/cosmetic.py::bulk_assign` ) | `admin` or `cosmetic_manager` only | Tag in `BULK_ASSIGN_STAGES` | New WorkID per tag. Other cosmetic stages: reassignment only, no move |
| T33 | `cosmetic_received` , `cleaning` , `water_sanding` , `cosmetic_completed` | `l1` | Cosmetic → Fail ( `routers/cosmetic.py::cosmetic_fail_assign` ) | Per-stage edit permission · Cosmetic Lead | Only these 4 stages; engineer must be `l1_engineer` / `l2_engineer` | `repair_notes` = " Failed — ..."; status reset; `l1` WorkID; notification; audit `COSMETIC_FAIL_ASSIGNED` |
| T34 | `final_qc` | `final_qc_pass_hold` | Final QC → Pass ( `routers/cosmetic.py::advance_stage` ) | `cosmetic_finalqc` edit · Final QC Inspector | — | Completed `fqc` WorkID for the inspector; grade set; touch type; spec corrections; verified RAM/HDD changes written ( `apply_spec_changes_on_pass` ); **price finalised once** (base + parts consumed + changed parts); bucket by name |
| T35 | `final_qc` | `final_qc_fail_hold` | Final QC → Fail ( `advance_stage` ) | `cosmetic_finalqc` edit | — | `final_qc_status` = fail; `fqc_failure_reason` , `fqc_final_notes` ; bucket fail reason/engineer recorded |
| T36 | Pass/Fail Hold | `final_qc` | Edit ( `routers/cosmetic.py::fqc_edit_revert` ) | `cosmetic_finalqc` edit | Tag must be in a hold | Decision fields cleared |

| # | From | To | Trigger / screen | Role | Guards | Side effects |
|---|---|---|---|---|---|---|
| T37 | `final_qc_pass_hold` | `ready_to_sale` | Devices Passed → Move to Inventory ( `routers/cosmetic.py::fqc_move_passed` ) | `cosmetic_finalqc` edit · Final QC Inspector → Inventory Manager | Tag must be at Pass Hold | StageMovement; default location |
| T38 | `final_qc_fail_hold` | `l1` / `trc_production` / `qc_check` / `cosmetic_received` | Devices Failed → Move / Bulk Move ( `routers/cosmetic.py::fqc_move_failed` ) | `cosmetic_finalqc` edit | Tag at Fail Hold and reason is Hardware, Software or Cosmetic | Routed by `FAIL_REASON_TO_DEPT` ; to last engineer at that step ( `_resolve_fail_engineer` ). Hardware with no engineer → `trc_production` . Uses `_apply_department_move` (T12 side effects) |

## 4.5 Sales, returns and scrap

| # | From | To | Trigger / screen | Role | Guards | Side effects |
|---|---|---|---|---|---|---|
| T39 | **any stage** | `sold` | New Sale / New Tag Sale / Ready to Sale bulk / Sell from Block ( `routers/sales.py::create_sale` ) | `admin` , `sales` , `sales_manager` , `telecaller` + `sales` add · Sales Executive | Tag entity = seller's `User.entity` (seller must have one); not blocked unless sold from its own block ( `validate_sale_allowed` ) | Sale row `SALE-nnnn` ; company snapshot; warranty expiry; sale invoice no. replaces tag invoice no.; block item released with note "Sold — SALE-..."; below-cost warning (not blocking); audit `SALE_CREATED` ; event `SALE_COMPLETED` |
| T40 | any | any | Customise modal → Move to Stage ( `routers/iqc.py::iqc_bulk_apply_grade_type` ) | IQC roles + `iqc` or `devices` edit | `sold` refused. `validate_transition` skipped for admin and when posted from `/devices` or `/stock` | Status reset on L1/L2 entry; L1/L2 or L3/L4 WorkOrders closed on exit; spec changes past Final QC |
| T41 | any | any (allowed) | Manual move ( `routers/repair.py::move_device` ) | Any user | `validate_transition` (admin bypass). **Does not refuse `sold`** | Closes `L3L4-` WorkOrders when leaving `l3` ; audit `STAGE_MOVED` |
| T42 | any | any (allowed) | API v1 ( `routers/api_v1/devices.py::move_device_stage` ) | API key | Transition must exist | StageMovement by `api_key:<name>` |
| T43 | any | any (allowed) | Bulk upload duplicate → Approve Move ( `routers/bulk_upload.py::approve_duplicate_move` ) | Bulk upload roles | `validate_transition` | audit `STAGE_MOVED` |

| # | From | To | Trigger / screen | Role | Guards | Side effects |
|---|---|---|---|---|---|---|
| T44 | `sold` (or any with a sale) | `stock_in` | Return New → Internal Tag ( `routers/sales.py::process_return` ) | Sales roles + `returns` add · Sales Executive | A sale must exist (or tag at `sold` → placeholder sale); one return per sale; reason, condition, received-by required | Return row `pending` ; `return_status` = true; `tag_return_status` = "Return for Repair" or "Replaced by "; audit `RETURN_SUBMITTED` |
| T45 | `ready_to_sale` | `sold` | Return New → Replace Now ( `process_return` ) | As T44 | Replacement tag must be `ready_to_sale` and not the same tag | ₹0 Sale with original customer; `replaced` = "Replaced with "; customer invoice no. copied; audit `RETURN_REPLACEMENT_SOLD` |
| T46 | any | `iqc` / `scrapped` | Pending Returns → Approve ( `routers/sales.py::approve_return` ) | `admin` , `sales_manager` only | Return must be `pending` | Restock → `iqc` ; Scrap → `scrapped` ; audit `RETURN_APPROVED` |
| T47 | — | — | Pending Returns → Reject ( `reject_return` ) | `admin` , `sales_manager` | Return must be `pending` | Status `rejected` . **Tag stays at `stock_in`** (Q2) |
| T48 | (new) | `iqc` | Return New → External Tag ( `process_external_return` ) | As T44 | Reason, condition, received-by required | New device in lot `EXT-RETURNS` if tag unknown; placeholder ₹0 sale; pending Return with paid-repair amount |
| T49 | any (returned tag) | `sold` | Return Stock → Complete ( `routers/stock.py::return_stock_complete` ) | Stock roles · Inventory Manager | A return and its original sale must exist | New Sale with original customer and warranty days; `return_status` = false; audit; event `SALE_COMPLETED` |
| T50 | `sold` | previous stage | Sales list → Delete Selected ( `routers/sales.py::bulk_delete_sales` ) | `admin` only | Referenced sales (credit notes, receipts, care) skipped | Audit `SALE_DELETED` before delete; tag returns to the stage it was sold from |
| T51 | `sold` | `ready_to_sale` | Entity Movement → ReGRN ( `routers/entity_movement.py` ) | `entity_movement` permission | Tag not trashed | New GRN/invoice/entity; location; audit `REGRN_TAG` |
| T52 | any | `scrapped` | Cost engine auto-scrap ( `services/cost_engine.py::auto_scrap_device` , from `complete_repair` ) | System | Total cost ≥ expected sale value | AuditLog `AUTO_SCRAP` |

| # | From | To | Trigger / screen | Role | Guards | Side effects |
|---|---|---|---|---|---|---|
| T53 | any | `scrap_for_sale` | Scrap Products from Repair Line → Move to Inventory ( `routers/stock.py::scrap_move_to_inventory` ) | Stock roles | **No stage guard** | StageMovement only |
| T54 | `ready_to_sale` | `ready_to_sale` (blocked) | Block Tags → New Block ( `routers/block_tags.py::create_block` ) | `block_tags` add · Inventory Manager, Sales Executive | Not already blocked. Ready-to-Sale check is in the search box only (Q11) | `is_blocked` = true; BlockRecord `DDMMYYYY-CONTAINER-0001` ; `assigned_to_user_id` = Block For user. **No audit** |
| T55 | blocked | open | Block Tags → Release / Bulk Release ( `_release_items` ) | `block_tags` edit | Item still `blocked` | `is_blocked` = false; item `released` |

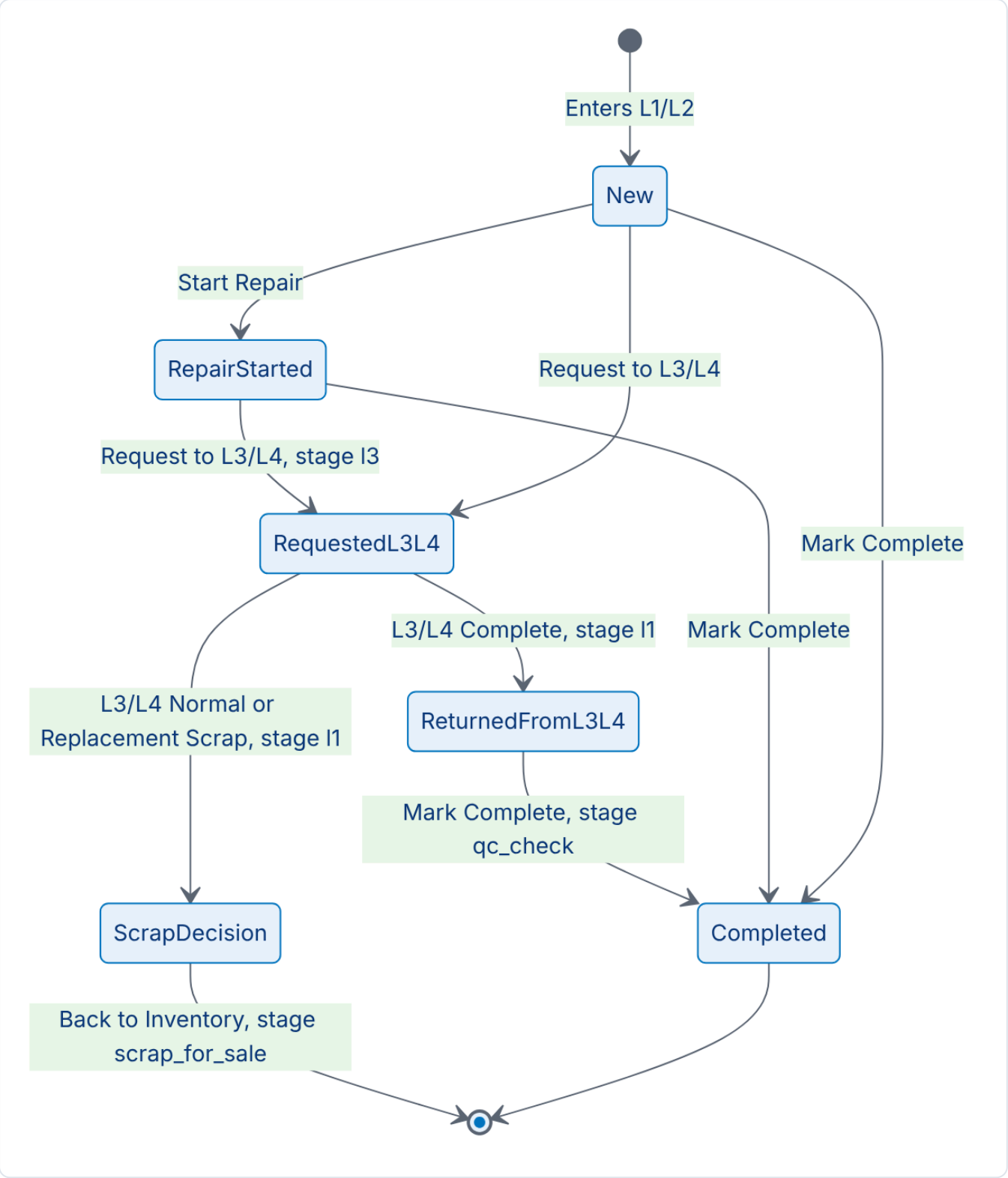
## 5. Flags and status fields beside the stage

| Field | Values | Set by | Cleared by |
|---|---|---|---|
| `is_blocked` | true / false | `block_tags.py::create_block` | `_release_items` ; sale from block |
| `flag_*` (Urgent flag) | 4 required fields | `stock.py::stock_flag_save` | Remove action; automatically on entry to `stock_in` ( `models/device.py::_clear_flag_on_stock_in` ) |
| `return_status` | true / false | Any return or credit note | `return_stock_complete` ; last Return deleted |
| `tag_return_status` | "Return for Repair", "Replaced by ", "Return for Credit Note" | `process_return` , `process_credit_note` | Last Return deleted |
| `final_qc_status` | pass / fail | Final QC decision | Edit (revert) |
| `grade` | A, B, C, D, E, scrap ( `DeviceGrade` ) | IQC form, Customise, Final QC Pass, Back to Inventory (scrap) | — |
| `scrap_verified` | true / false | `scrap.py::verify_scrap` | — |
| `sub_lot_number` | free text | GRN "Yes As-Is", transfers As-Is Lot, Customise | Customise Stock Type Finish Good / In Progress |

# 6. Sub-state machines

## 6.1 L1/L2 and L3/L4 repair status

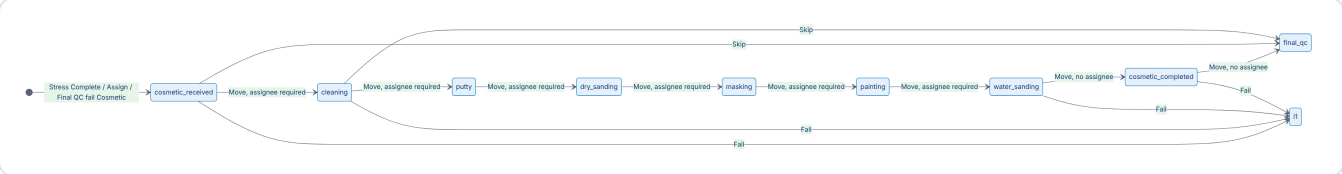
`devices.l1l2_status` and `devices.l34_status` (`models/device.py`). Both reset when a tag enters `l1` / `l2` from any other stage (`reset_repair_cycle_on_entry`).




`l34_status` values: blank → "Repair Started" → "Completed" or "Normal Scrap" / "Replacement Scrap".

## 6.2 Cosmetic chain

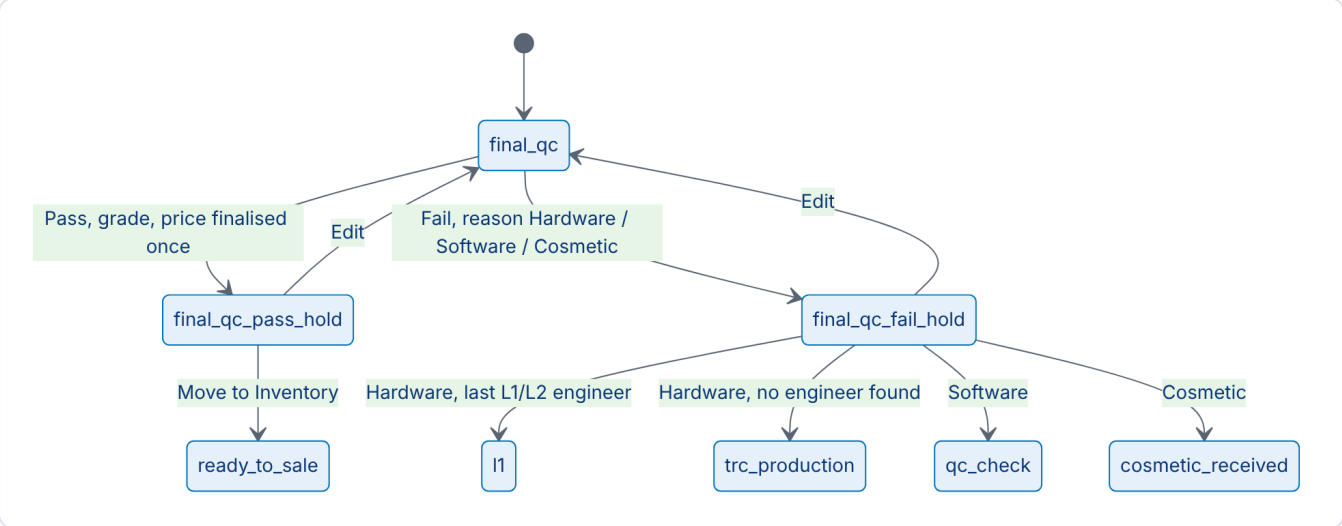
Order is fixed in `routers/cosmetic.py::COSMETIC_PIPELINE` and `NEXT_COSMETIC`.




| Stage | WorkID code | Permission module | Fail allowed | Bulk Assign |
|---|---|---|---|---|
| Cosmetic Received | `recv` | `cosmetic_received` | Yes | Yes (also moves to Cleaning) |
| Cleaning | `clean` | `cosmetic_cleaning` | Yes | Yes (reassign only) |
| Putty | `putty` | `cosmetic_putty` | No | Yes (reassign only) |
| Dry Sanding | `drysd` | `cosmetic_dry_sanding` | No | Yes (reassign only) |
| Masking | `mask` | `cosmetic_masking` | No | Yes (reassign only) |
| Painting | `paint` | `cosmetic_painting` | No | Yes (reassign only) |
| Water Sanding | `water` | `cosmetic_water_sanding` | Yes | Yes (reassign only) |
| Cosmetic Completed | `comp` | `cosmetic_completed` | Yes | No |

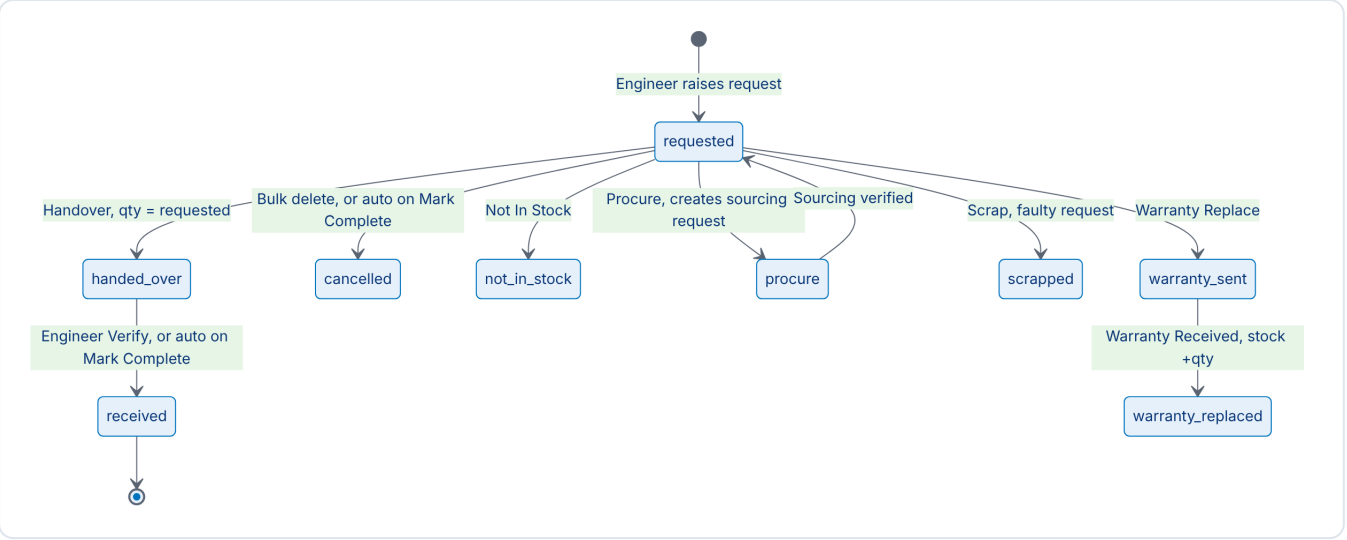
Skip to Final QC is allowed from any cosmetic stage (`advance_stage`, `target=final_qc`). Flow Data rows (`models/cosmetic_flow.py::CosmeticFlowRow`) name one user per stage; when the mover sits in a row, the next user is picked automatically.

## 6.3 Final QC holds




## 6.4 Part request and PNA

`part_requests.status` (`models/part_request.py::PartRequest`). Actions in `routers/part_requests.py` and `routers/repair.py`.




| Action | Endpoint | Role | Guard |
|---|---|---|---|
| Raise (single / multi / bulk) | `create_part_request`, `create_part_requests_batch`, `repair.py::bulk_part_request` | `admin`, `inventory_manager`, `l1/l2/l3_engineer` (bulk: any user) | Bulk skips tags with a live request for the same part |
| Handover | `handover_part` | `admin`, `spare_parts_manager` | None on current status (single); bulk only from `requested` |
| Verify receiving | `validate_receiving` | Any user | None |
| Not In Stock / Procure / Scrap / Warranty Replace | `not_in_stock`, `procure_part`, `scrap_part_request`, `warranty_replace` | `admin`, `spare_parts_manager` | None on current status |
| Warranty Received | `warranty_received` | `admin`, `spare_parts_manager` | Must be `warranty_sent` |

**Sourcing request** (`PartSourcingRequest.status`): `open` → `closed` (Sales Manager, `close_sourcing`) → verified (Spare Parts Manager, `verify_sourcing`, needs `closed`; adds stock via `SparePartsLedger` IN). Production requests from Part Estimation (`source = production`) are confirmed instead (`confirm_production_sourcing`).

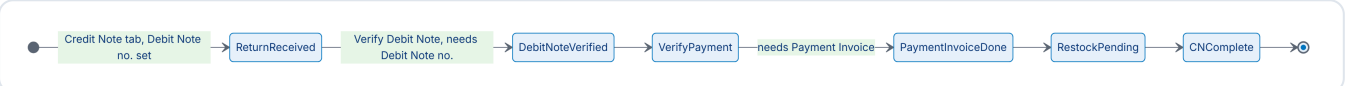
**PNA** (`models/pna_part.py::DevicePNAPart`): one row per tag + part. Mark sets `is_active` = true; clear sets false and keeps the row (`mark_part_pna`, `clear_all_pna`; any signed-in user). While any PNA row is active, L1/L2 Mark Complete is refused (T23).

## 6.5 Return approval

`returns.approval_status`: `pending` → `approved` (tag to `iqc` or `scrapped`) or `rejected` (no stage change). Only `admin` and `sales_manager` (`routers/sales.py::MANAGER_ROLES`).

## 6.6 Credit note

`returns.cn_stage` (`models/sales.py::CN_STAGES`). It never changes the device stage.



- Entry: `routers/sales.py::process_credit_note` (sales roles + `returns` add). It creates a Return and placeholder ₹0 sale if the tag has none. Re-submitting never moves a record back.
- Advance: one step at a time, `credit_note_advance` or `credit_note_bulk_verify` (`credit_note` edit permission), rules in `_try_advance_return`.

### 6.7 Block item, WorkOrder and delivery

| Machine | States | Source |
|---|---|---|
| Block item | `blocked` → `released` (by Release or by sale from block) | `models/block_tags.py::BlockItemStatus` |
| WorkOrder | `pending` → `in_progress` (L3/L4 start) → `completed` | `models/work_order.py` |
| Sale delivery | `pending` → `dispatched` (Gate Pass exit) → `delivered` | `models/sales.py::Sale.delivery_status`, `routers/gate_pass.py::confirm_exit` |
| Return Stock repair cost | Verify records repair + labour cost, Complete re-sells | `routers/stock.py::return_stock_verify`, `return_stock_complete` |

## 7. Invariants

These hold in the code today, with the exceptions named.

| # | Invariant | Enforced by | Known exceptions |
|---|---|---|---|
| I1 | A tag reaches `sold` only with a Sale row | `create_sale`, `process_return` (Replace Now), `return_stock_complete`; `sold` is not in `DROPDOWN_STAGES`; Customise refuses `sold` | `repair.py::move_device` and `api_v1/devices.py::move_device_stage` accept `sold` if `ready_to_sale → sold` is in `allowed_transitions` (Q3). `process_return` makes a placeholder sale for old orphans |
| I2 | A blocked tag cannot be sold, except from its own block | `control_engine.validate_sale_allowed`, `create_sale::_check_sellable` | — |
| I3 | A seller can only sell tags of their own entity | `create_sale::_check_sellable` | `return_stock_complete` and Replace Now do not check entity |
| I4 | One return per sale | `process_return` | External Tag and Credit Note create their own placeholder sale |
| I5 | Every stage move writes a StageMovement | Each endpoint | `stock.py::advance_grn_to_iqc`, `buckets.py::create_bucket` (Q12) |
| I6 | Entry into L1/L2 from another stage starts a fresh repair cycle | `models/device.py::reset_repair_cycle_on_entry`; Stress/Cosmetic fail set it directly | `repair.py::move_device` does not call it |
| I7 | A tag with active PNA parts cannot be Marked Complete in L1/L2 | `l1l2_complete_to_stress` | Legacy `complete_repair` and Customise moves do not check PNA |
| I8 | The Final QC price is finalised once per tag | `advance_stage` checks for an earlier "price finalized" StageMovement | — |
| I9 | Entering `stock_in` clears the urgent flag | SQLAlchemy listener `_clear_flag_on_stock_in` | Bulk `update()` statements bypass ORM listeners |

| # | Invariant | Enforced by | Known exceptions |
|---|---|---|---|
| I10 | Credit Note stages only move forward, one at a time | `_try_advance_return` | Delete removes the record |
| I11 | Leaving L1/L2 closes the L1/L2 WorkIDs | `_close_l1l2_work_orders`, `close_open_work_orders` | Stress Complete/Fail do not close the open `STRS-` WorkID (Q10) |

## 8. Allowed-transitions table

The live table is edited by admins on `/stage-control` (`routers/stage_control.py::add_transition`, `remove_transition`). Three seed lists exist and differ:

| Source | Includes cosmetic_received / cosmetic_completed / putty? | Notes |
|---|---|---|
| `models/stage_control.py::DEFAULT_TRANSITIONS` | No | Has `qc_check → cleaning`, `final_qc → cleaning` |
| `upgrade_db.py` (CORRECT_TRANSITIONS) | No | Smaller list |
| `db_validator.py::CANONICAL_TRANSITIONS` | cosmetic_received and cosmetic_completed yes; putty no | Has fail moves to `l1` |

None of the three includes `putty`, `final_qc_pass_hold`, `final_qc_fail_hold`, `scrap_for_sale` or `grn`.

## 9. Open questions / gaps

| # | Gap | Code fact | Business view | Suggested owner |
|---|---|---|---|---|
| Q1 | Sale from any stage | `create_sale` sells from any stage, including `sold`, `scrapped` and in-repair stages (decision dated 19 Oct 2026 in `control_engine.validate_sale_allowed`). A sold tag can be sold again | Sales Executive sells **Ready for Sale** items | Director / Sales |
| Q2 | `returned` stage unused; reject leaves tag in Stock In | `process_return` moves the tag to `stock_in` at once; approve moves to `iqc` / `scrapped`; reject leaves it at `stock_in` though the code comment says "Device stays sold" | Returns: return, paid repair, replacement, credit note | Sales Manager |
| Q3 | Transition rules are weak | Three seed lists differ (section 8); most workflow screens skip `validate_transition`; Manual Move and API can set `sold` | Clear, enforced stage flow | Architect |
| Q4 | Roles in code ≠ 12 business roles | No built-in Production Manager, Final QC Inspector, Accounts or TRC Manager role. Production Manager = `inventory_manager` (`routers/buckets.py::production_manager`). Final QC = anyone with `cosmetic_finalqc` edit. Custom roles work through the permission matrix (`auth/dependencies.py::require_roles`) | 12 distinct roles | Admin |
| Q5 | Many moves have no stage guard and only need a login | `l1l2_complete_to_stress`, `back_to_production`, `l3l4_complete`, `l3l4_scrap`, `stress_fail_assign`, `stress_complete_to_paint`, `move_to_stock`, `send_to_cosmetic`, `scrap_move_to_inventory` | Only the owner role moves a tag forward | Architect |

| # | Gap | Code fact | Business view | Suggested owner |
|---|---|---|---|---|
| Q6 | Scrap sale does not move tags | `scrap_for_sale.py::sell_scrap_for_sale` marks the batch sold; tags stay at `scrap_for_sale`. Two scrap stages ( `scrapped` , `scrap_for_sale` ) | Tags scrapped handled by Inventory Manager | Inventory |
| Q7 | Customise bypass depends on a form field | `iqc_bulk_apply_grade_type` skips transition checks when the posted `return_to` is `/devices` or `/stock` | — | Architect |
| Q8 | IQC Inspector cannot run GRN in code | `routers/grn.py` allows only `admin` , `inventory_manager` | IQC Inspector owns GRN, Lot, IQC | Admin |
| Q9 | Two grade scales | `QCCheck.grade` A/B/C/D/S ( `models/qc.py` ) vs `DeviceGrade` A–E/scrap | Final QC assigns grading | Final QC |
| Q10 | Open `STRS-` WorkID after Stress | Stress Complete/Fail create new WorkIDs but do not close the `STRS-` one raised by Mark Complete | Productivity reporting | Production |
| Q11 | Block create trusts the client | `create_block` does not re-check `ready_to_sale` ; no audit on block/release; model comment mentions a "Done" status the enum lacks | Block tags for any staff | Inventory / Sales |
| Q12 | Silent stage moves | `advance_grn_to_iqc` and `create_bucket` change stage without StageMovement; `create_bucket` moves any stage to `stock_in` | Full history | Architect |
| Q13 | Rentals | No rental flow found in sales, dispatch or gate pass | Sales Executive handles rentals | Sales |